Repaso Parcial

Estado	En curso
Tipo	Libro

Spring:

Inyeccion Dependencias

1. ARQUITECTURA QUE SIEMPRE USAS EN EL PARCIAL

Siempre piensa en 4 capas:

```
model/
repository/
service/
servlet/
config/
```

📌 Model

Representa datos.

NO tiene lógica fuerte.

📌 Repository

Hace CRUD en memoria (List).

📌 Service

Contiene reglas de negocio.

Coordina repositorios.

📌 Servlet

Recibe request, llama al service, manda respuesta.

🧱 2. MODELOS BIEN HECHOS

Reglas obligatorias:

- Atributos privados
- Constructor
- Getters y setters
- Listas inicializadas

Ejemplo mental genérico:

```
publicclassEntity {

privateLongid;
privateStringname;
privateList<OtherEntity>related=newArrayList<>();

publicEntity(Longid,Stringname) {
this.id=id;
this.name=name;
    }

// getters y setters
}
```

Si hay relación many-to-many:

En ambas clases hay lista.

Y cuando agregas relación:

```
a.getListB().add(b);
b.getListA().add(a);
```

Si no actualizas ambos lados → pierdes puntos.

🗂️ 3. REPOSITORY (CRUD EN MEMORIA)

Siempre tiene:

```
privateList<Entity>entities=newArrayList<>();
```

Métodos típicos:

```
List<Entity>findAll();
voidsave(Entitye);
EntityfindById(Longid);
voiddeleteById(Longid);
EntityfindByName(Stringname);
```

Eliminar bien:

```
entities.removeIf(e ->e.getId().equals(id));
```

Repository NO hace reglas.

🧠 4. SERVICE (LA PARTE IMPORTANTE)

Aquí va:

- Validaciones
- Crear relaciones
- Buscar cosas complejas
- Coordinación entre repositorios

Ejemplo:

```
publicvoidassignRelation(LongidA,LongidB){

Aa=aRepository.findById(idA);
Bb=bRepository.findById(idB);

a.getListB().add(b);
```

```
b.getListA().add(a);
}
```

Service depende de Repository.

Pero no lo crea.

Y aquí entra DI.

💉 5. INYECCIÓN DE DEPENDENCIAS (LO QUE MÁS PREGUNTAN)

🎯 Qué es

Es cuando un objeto recibe sus dependencias en vez de crearlas.

Mal:

```
privateRepositoryrepository=newRepository();
```

Bien:

```
privateRepositoryrepository;

publicService(Repositoryrepository){
this.repository=repository;
}
```

Eso es constructor injection.

🔥 6. FORMAS DE CONFIGURAR SPRING

IMPORTANTE:

Constructor injection es el método.

XML / Annotations / JavaConfig es la configuración.

🟢 A) XML CONFIGURATION

1. Clase normal

```
publicclassService {

privateRepositoryrepository;

publicService(Repositoryrepository){
this.repository=repository;
    }
}
```

2. beans.xml

```
<beanid="repository"
class="com.example.Repository"/>

<beanid="service"
class="com.example.Service">
<constructor-argref="repository"/>
</bean>
```

Qué hace:

- Crea repository
- Crea service
- Le pasa repository al constructor

🟢 B) ANNOTATIONS

Aquí usas anotaciones en las clases.

📌 @Component

Genérica. Marca clase como bean.

📌 @Repository

Para repositorios.

📌 @Service

Para servicios.

📌 @Controller

Para controladores (si usaras Spring MVC).

Ejemplo:

```
@Repository
publicclassRepository { }
```

```
@Service
publicclassService {

privateRepositoryrepository;

    @Autowired
publicService(Repositoryrepository){
this.repository=repository;
    }
}
```

Y en XML solo activas:

```
<context:component-scanbase-package="com.example"/>
```

Qué pasa:

Spring escanea el paquete.

Encuentra clases con anotaciones.

Las crea automáticamente.

Resuelve el constructor.

📌 @Autowired

Se usa para decirle a Spring:

"Inyecta esta dependencia automáticamente"

Puede ir:

- En constructor (mejor práctica)
- En atributo (menos recomendado)
- En setter

Ejemplo en atributo (menos recomendado):

```
@Autowired
privateRepositoryrepository;
```

Mejor siempre en constructor.

🟢 C) JAVA CONFIGURATION

Sin XML.

```
@Configuration
publicclassAppConfig {

    @Bean
publicRepositoryrepository(){
returnnewRepository();
    }

    @Bean
publicServiceservice(){
returnnewService(repository());
    }
}
```

📌 @Configuration

Marca clase como clase de configuración.

📌 @Bean

Le dice a Spring:

"Este método crea un bean".

🧠 7. DIFERENCIA CLAVE

Concepto	Qué es
IoC	Spring controla creación de objetos
DI	Spring inyecta dependencias
Bean	Objeto gestionado por Spring

🌱 8. INICIALIZACIÓN DE DATOS

Nunca en servlet.

Con annotations:

```
@PostConstruct
publicvoidloadData(){
// crear datos iniciales
}
```

📌 @PostConstruct

Se ejecuta después de que Spring crea el bean.

Muy útil para cargar datos iniciales.

🌐 9. SERVLETS EN EXAMEN

En servlet:

```
ApplicationContextcontext=
newClassPathXmlApplicationContext("beans.xml");

Serviceservice=context.getBean(Service.class);

List<Entity>list=service.findAll();
```

```
request.setAttribute("list",list);
request.getRequestDispatcher("list.jsp")
.forward(request,response);
```

El servlet solo:

- Obtiene datos del request
- Llama service
- Manda datos a JSP

Nunca hace lógica fuerte.

Hay tres formas de injection (las mismas tres versiones del taller):

1️⃣ Constructor injection (la mejor práctica)

```
private ArtistRepository artistRepository;

public ArtistService(ArtistRepository artistRepository) {
                this.artistRepository=artistRepository;
}
```

Spring crea ambos objetos y le pasa el repository al constructor.

Esto es lo más limpio y profesional.

2️⃣ Setter injection (más común en XML)

```
privateArtistRepositoryartistRepository;

publicvoidsetArtistRepository(ArtistRepositoryartistReposit
ory) {
this.artistRepository=artistRepository;
}
```

Spring crea el objeto y luego llama al setter.

3️⃣ Field injection (menos recomendado)

```
@Autowired
private ArtistRepository artistRepository;
```

Funciona, pero no es buena práctica en proyectos grandes.

📋 ESCENARIO DE EJEMPLO de borrado rapido de una lista con codigo compacto

Primero, definamos una clase de ejemplo:

java

```
class Producto {
    private int id;
    private String nombre;
    private double precio;

    public Producto(int id, String nombre, double precio) {
        this.id = id;
        this.nombre = nombre;
        this.precio = precio;
    }

    // Getters y setters
    public int getId() { return id; }
    public String getNombre() { return nombre; }
    public double getPrecio() { return precio; }

    @Override
    public String toString() {
        return "Producto{id=" + id + ", nombre='" + nombre
+ "', precio=" + precio + "}";
    }
}
```

Y una lista de productos:

java

```
List<Producto> productos = new ArrayList<>(Arrays.asList(
    new Producto(1, "Laptop", 1200.0),
    new Producto(2, "Mouse", 25.0),
    new Producto(3, "Teclado", 80.0),
    new Producto(4, "Monitor", 350.0),
    new Producto(5, "Audífonos", 60.0)
));
```

Queremos eliminar el producto con `id = 3`.

🔷 MÉTODO 1: removeIf (El más RECOMENDADO y elegante)

java

```
boolean eliminado = productos.removeIf(p -> p.getId() ==
3);

System.out.println("¿Eliminado? " + eliminado);
System.out.println("Lista actualizada: " + productos);
```

📝 Explicación detallada:

- `removeIf` recorre la lista y elimina TODOS los elementos que cumplan la condición
- `p -> p.getId() == 3` es un lambda que dice: "para cada producto p, si su id es 3, elimínalo"
- Devuelve `true` si eliminó al menos un elemento

Salida:

text

```
¿Eliminado? true
Lista actualizada: [Producto{id=1, nombre='Laptop'...}, Pro
```

```
ducto{id=2, nombre='Mouse'...},
                     Producto{id=4, nombre='Monitor'...}, Pr
oducto{id=5, nombre='Audífonos'...}]
```

🎯 Ventajas:

- ✅ Una sola línea de código
- ✅ Modifica la lista original
- ✅ Muy legible
- ✅ Maneja múltiples ocurrencias (si hay varios con mismo ID)

🔷 MÉTODO 2: Buscar con stream y luego eliminar (Más control)

java

```
// PASO 1: Buscar el producto por ID usando stream
Producto productoAEliminar = productos.stream()
    .filter(p -> p.getId() == 3)        // Filtra por ID
    .findFirst()                         // Toma el primero
que encuentra
    .orElse(null);                       // Si no existe, d
evuelve null

// PASO 2: Verificar si existe y eliminarlo
if (productoAEliminar != null) {
    productos.remove(productoAEliminar);
    System.out.println("Producto eliminado: " + productoAEl
iminar);
} else {
    System.out.println("No se encontró producto con ID 3");
}

System.out.println("Lista actualizada: " + productos);
```

📝 Versión más compacta con ifPresent:

java

```
productos.stream()
    .filter(p -> p.getId() == 3)
    .findFirst()
    .ifPresent(productos::remove);  // Si existe, lo elimin
a
```